

LightCCI 架构微调方案 V2

sparse_cci、sparse_cci_grn 与 joint_cci_grn

三个新增网络结构的数学定义与代码级实施计划

固定 N (Joint NMF) + KL 主线，分别验证 CCI 稀疏化、GRN 状态回填与联合分解

本版计划明确新增三个相互独立、逐级验证的网络结构：sparse_cci 只处理 CCI 过密；sparse_cci_grn 在稀疏 CCI 上加入由双端表达门控得到的 unit-level GRN 状态，并在 KL cost 层与 CCI-NMF 信息融合；joint_cci_grn 则让 GRN 先经过有向 Joint NMF，再通过表达矩阵映射到 unit 空间，并与稀疏 CCI 一起进入 Collective Joint NMF。三条路线均保留原始 light_cci 作为不可修改的基线。

生成日期：2026 年 7 月 13 日

项目：2026 因果涌现

文档类型：项目书 / Codex 实施计划

目录

1 本轮架构总览	3
1.1 一个基线与三个新增网络	3
1.2 固定的 PIJ 主线	3
1.3 验证顺序	3
2 新增网络一：sparse_cci	3
2.1 为什么先单独处理 CCI 稠密问题	3
2.2 sparse_cci 的数学定义	4
2.3 建议默认参数	4
2.4 代码结构	5
2.5 必须输出的诊断	5
3 新增网络二：sparse_cci_grn	5
3.1 网络语义	5
3.2 输入与维度	6
3.3 保留双端表达门控，不改成纯传播特征	6
3.4 固定随机投影到 64 维	7
3.5 为什么不再把 N 与 G 直接拼接后统一 softmax	7
3.6 代码结构	7
3.7 domain 层的 GRN 状态来源	8
4 新增网络三：joint_cci_grn	8
4.1 这个网络想解决什么问题	8
4.2 符号、矩阵与维度总表	9
4.3 步骤 0：对齐 gene universe	9
4.4 步骤 1：对 GRN 做 pairwise 有向 Joint NMF	10
4.5 步骤 2：用表达把 gene module 映射到 unit，并保留双端门控	10
4.6 步骤 3：CCI 与 unit-level GRN state 做 Collective Joint NMF	11
4.7 为什么这个目标能实现“GRN 经过 Joint NMF”	12
4.8 最终 joint-N 特征与 KL	12
4.9 一个具体形状例子	13
4.10 pairwise 语义与信息泄露控制	13
4.11 代码结构	14
4.12 优化与初始化要求	15
5 三个网络的实验矩阵	15
5.1 正式对比	15
5.2 阶段化运行	15

6	配置字段建议	16
6.1	sparse CCI	16
6.2	sparse_cci_grn	16
6.3	joint_cci_grn	16
7	测试计划	16
7.1	sparse_cci	17
7.2	sparse_cci_grn	17
7.3	joint_cci_grn	17
7.4	集成 smoke test	17
8	Codex 不得做的事情	18
9	完成标准	18
10	最终算法一句话	19

1. 本轮架构总览

1.1 一个基线与三个新增网络

本轮不把全部改动混在一个网络名称下，而是保留清晰的实验分支：

```
light_cci          # 原始基线，不改变
|
|-- sparse_cci      # 新网络 1: 只解决 CCI 过密
|   |-- sparse_cci_grn # 新网络 2: 路线二，NMF 后融合 GRN 状态
|   |-- joint_cci_grn  # 新网络 3: CCI 与 GRN 共同参与联合分解
```

三者的核心区别如下。

网络	CCI 处理	GRN 进入位置	最终进入 KL 的特征
sparse_cci	CCI 先做 top-k 稀疏化、log 压缩和可选行归一化	非 gene 层不额外加入 GRN；gene 层沿用原 GRN	稀疏 CCI 的 N 特征
sparse_cci_grn	与 sparse_cci 相同	GRN 通过双端表达门控形成 unit 状态，独立于 CCI-NMF	分块 KL 融合后的 N-cost 与 G-cost
joint_cci_grn	与 sparse_cci 相同	GRN 先做有向 Joint NMF，再通过表达桥映射到 unit，并约束 CCI 的 Collective Joint NMF	同时受 CCI 与 GRN 约束的 joint-N 特征

1.2 固定的 PIJ 主线

所有正式对比均固定使用：

$$N \text{ (Joint NMF 类特征)} + \text{KL cost} \rightarrow P_{ij} \rightarrow EI$$

因此，本轮关注的是网络输入与 N 的形成方式，而不是再次展开 cosine、Euclidean、SOT、L、E、SR 的全组合。

1.3 验证顺序

1. 先比较 light_cci 与 sparse_cci，确认 CCI 稀疏化是否改善 EI；
2. 再比较 sparse_cci 与 sparse_cci_grn，确认独立 GRN 状态是否提供增益；
3. 最后比较 sparse_cci_grn 与 joint_cci_grn，判断让 GRN 真正进入 Joint NMF 是否进一步改善结果。

2. 新增网络一：sparse_cci

2.1 为什么先单独处理 CCI 稠密问题

当前 LightCCI 的非 gene 层主要使用 CCI total matrix。domain 层将大量 spot-to-spot 通信累加到 domain-to-domain 后，几乎任意两组 domain 都可能得到非零权重。问题不只是“非零比例高”，

而是高密度与强公共背景共同导致不同 unit 的通信轮廓趋同：

$$A_{\text{CCI}} \approx A_{\text{global}} + \Delta A_{\text{unit}}.$$

若 A_{global} 占主导，Joint NMF 首先解释的是公共通信背景，unit-specific 变化只落在较小残差中。随后不同 source unit 的 KL cost 行可能变得接近，最终：

$$P_{1,:} \approx P_{2,:} \approx \dots \approx P_{n,:}.$$

而 EI 可以写成：

$$EI(P) = \frac{1}{n} \sum_i D_{\text{KL}}(P_{i,:} \| \bar{P}), \quad \bar{P} = \frac{1}{n} \sum_i P_{i,:}.$$

当各行差异不足时，EI 会下降；若 upper 层比 lower 层更同质，就会出现 $EI_{\text{gain}} < 0$ 。因此，sparse_cci 的目标不是主观地追求更多零，而是削弱低信息量公共边，让 NMF 更集中于通信偏好。

2.2 sparse_cci 的数学定义

对每个非 gene 层，输入 CCI 邻接矩阵：

$$A_{\text{raw}} \in \mathbb{R}_{\geq 0}^{U \times U}.$$

先做 log 压缩：

$$A'_{ij} = \log(1 + A_{ij}).$$

再同时保留每一行的 outgoing top-k 与每一列的 incoming top-k：

$$\mathcal{E} = \text{TopK}_{\text{row}}(A') \cup \text{TopK}_{\text{col}}(A').$$

最终：

$$A_{ij}^{\text{sparse}} = \begin{cases} A'_{ij}, & (i, j) \in \mathcal{E}, \\ 0, & \text{otherwise.} \end{cases}$$

可进行归一化：

$$\tilde{A}_{ij}^{\text{sparse}} = \frac{A_{ij}^{\text{sparse}}}{\sum_j A_{ij}^{\text{sparse}} + \epsilon}.$$

2.3 建议默认参数

参数	第一轮默认值	说明
cci_sparse_topk_out	10	每个发送 unit 保留 top-k outgoing CCI 边。
cci_sparse_topk_in	10	每个接收 unit 保留 top-k incoming CCI 边。

<code>cci_sparse_loglp</code>	<code>true</code>	压缩极端大权重，避免总通信强度支配 NMF。
<code>cci_sparse_row_normalize</code>	<code>true</code>	降低 <code>unit size / domain size</code> 对特征的影响。
<code>cci_sparse_keep_self</code>	<code>false</code>	<code>self-loop</code> 不作为普通跨 <code>unit</code> 通信边。

k40 可补充测试 $k = 5$ ，k150 和 spot 层第一轮统一使用 $k = 10$ ，避免一开始扩大参数网格。

2.4 代码结构

新增：

```
mignet_ce/networks/sparse_cci.py
```

推荐复用现有 LightCCI builder，只覆盖 CCI 邻接预处理：

```
def sparsify_cci_adjacency(
    matrix: sp.spmatrix,
    *,
    topk_out: int,
    topk_in: int,
    loglp: bool = True,
    row_normalize: bool = True,
    keep_self: bool = False,
) -> tuple[sp.csr_matrix, dict[str, object]]:
    ...
```

网络注册：

```
NETWORK_BUILDERS = {
    ...,
    "sparse_cci": SparseCCINetworkBuilder,
}
```

2.5 必须输出的诊断

1. raw 与 sparse 的 shape、nnz、density；
2. 每行非零数的 min、median、max；
3. 稀疏化前后 CCI 行间 cosine similarity；
4. 稀疏化前后第一 NMF 成分占比与 NMF 行方差；
5. KL cost 的 5/50/95 分位数；
6. PIJ 行熵分布；
7. mean/median EI_{gain} 与 $\Pr(EI_{gain} > 0)$ 。

3. 新增网络二：sparse_cci_grn

3.1 网络语义

`sparse_cci_grn` 是一个独立网络结构，不只是给 `sparse_cci` 临时换一个 PIJ 方法。它应当在每个 layer graph 中同时携带：

- 稀疏 CCI 邻接矩阵 A^{sparse} ;
- 与该层 unit 顺序严格对齐的 GRN 状态矩阵 G^{64} ;
- gene name、unit mapping、GRN coverage 和投影元数据。

其 PIJ 特征阶段仍然先从 CCI 得到 N，再读取网络自带的 G，并在 KL cost 层融合两种信息。

3.2 输入与维度

设：

- U ：当前层 unit 数；
- G ：进入 GRN 的基因数，典型值约 2000；
- $X \in \mathbb{R}_{\geq 0}^{U \times G}$ ：unit-by-gene 非负表达矩阵；
- $W \in \mathbb{R}_{\geq 0}^{G \times G}$ ：有向 GRN 权重矩阵， W_{ij} 表示 regulator i 指向 target j 。

每个 regulator 先保留 top-k target，再做出边归一化：

$$\widetilde{W}_{ij} = \frac{W_{ij}}{\sum_j W_{ij} + \epsilon}.$$

3.3 保留双端表达门控，不改成纯传播特征

对 unit u ，令其表达列向量为 $x_u \in \mathbb{R}_{\geq 0}^G$ 。定义：

$$g_u^{\text{reg}} = x_u \odot (\widetilde{W} x_u) \in \mathbb{R}_{\geq 0}^G,$$

$$g_u^{\text{tar}} = x_u \odot (\widetilde{W}^T x_u) \in \mathbb{R}_{\geq 0}^G.$$

这两个式子不是重复使用表达，而是在做双端表达门控。令 $D_u = \text{diag}(x_u)$ ，则 unit-specific active GRN 为：

$$W_u^{\text{active}} = D_u \widetilde{W} D_u, \quad (W_u^{\text{active}})_{ij} = x_{u,i} \widetilde{W}_{ij} x_{u,j}.$$

其行和正好是 regulator-side 状态：

$$W_u^{\text{active}} \mathbf{1} = x_u \odot (\widetilde{W} x_u),$$

其列和正好是 target-side 状态：

$$(W_u^{\text{active}})^T \mathbf{1} = x_u \odot (\widetilde{W}^T x_u).$$

因此，门控要求一条调控边的 regulator 端与 target 端都在当前 unit 中得到表达支持。它比仅使用 $\widetilde{W} x_u$ 更不容易把不同 unit 推向相同的全局传播模式；本版方案明确保留该门控，不将其替换为纯传播特征。

拼接后：

$$G_u^{2G} = [g_u^{\text{reg}}, g_u^{\text{tar}}] \in \mathbb{R}_{\geq 0}^{2G}.$$

当 $G = 2000$ 时，每个 unit 为 4000 维，但计算过程中不保存一张 $G \times G$ 的 unit-specific GRN。

3.4 固定随机投影到 64 维

使用由 (gene name, role, seed) 决定的固定随机投影：

$$R \in \mathbb{R}^{2G \times 64}, \quad G_u^{64} = G_u^{2G} R.$$

role 区分 regulator-side 和 target-side，确保同一个基因在两个角色中使用不同投影方向。实现采用 blockwise 流程：

$$X_{\text{block}} \rightarrow G_{\text{block}}^{2G} \rightarrow G_{\text{block}}^{64},$$

计算完成即丢弃中间的 $2G$ 维 block，避免长期保存 $U \times 4000$ 矩阵。

3.5 为什么不再把 N 与 G 直接拼接后统一 softmax

设 CCI-NMF 特征为 $N_u \in \mathbb{R}^{d_N}$ ，GRN 特征为 $G_u^{64} \in \mathbb{R}^{64}$ 。直接拼接虽然在形状上合法，但统一 softmax 会让两块竞争同一概率质量，贡献大小受列数、方差与数值尖锐度共同影响。为避免 CCI 继续压倒 GRN，本方案使用分块概率与分块 KL。

分别标准化后做 block-wise softmax：

$$p_u^N = \text{softmax}\left(\frac{\tilde{N}_u}{\tau_N}\right), \quad p_u^G = \text{softmax}\left(\frac{\tilde{G}_u^{64}}{\tau_G}\right).$$

分别形成 KL cost：

$$D_{uv}^N = D_{\text{KL}}(p_u^N \| p_v^N), \quad D_{uv}^G = D_{\text{KL}}(p_u^G \| p_v^G).$$

最终融合：

$$D_{uv}^{\text{NG}} = \lambda_N D_{uv}^N + \lambda_G D_{uv}^G, \quad \lambda_N + \lambda_G = 1.$$

第一轮固定：

$$\lambda_N = \lambda_G = 0.5.$$

这样，无论 d_N 与 64 是否相等，两块对最终 cost 的总权重都由 λ_N, λ_G 决定，而不是由列数决定。

3.6 代码结构

新增网络 builder：

```
mignet_ce/networks/sparse_cci_grn.py
```

新增 GRN 状态模块：

```
mignet_ce/features/grn_state.py
```

核心 API:

```
def build_gated_grn_state_features(
    expression_matrix,
    gene_names,
    grn_edges,
    *,
    topk_targets_per_regulator: int = 50,
    output_dim: int = 64,
    projection_seed: int = 20260713,
    block_size: int = 256,
) -> tuple[np.ndarray, dict[str, object]]:
    ...
```

新增 PIJ 方法:

```
mignet_ce/pij/compare/compare_N_G_block_kl.py
```

固定语义:

```
network = sparse_cci_grn
feature_blocks = ("N", "G")
pij_key = "block_kl"
fusion_mode = "weighted_block_kl"
```

3.7 domain 层的 GRN 状态来源

优先使用该层自身的 unit expression:

$$X^{(\ell)} \in \mathbb{R}^{U_\ell \times G}.$$

若某层没有可靠表达矩阵, 则从 spot-level gated GRN state 聚合:

$$G_d^{64} = \frac{\sum_{u \in d} w_{ud} G_u^{64}}{\sum_{u \in d} w_{ud} + \epsilon}.$$

元数据必须记录:

```
grn_state_source = "direct_layer_expression" | "aggregated_from_spot"
```

4. 新增网络三: joint_cci_grn

4.1 这个网络想解决什么问题

sparse_cci_grn 中, CCI 与 GRN 先各自形成特征, 直到 KL cost 层才融合。它清晰、容易实现, 但 GRN 并没有参与 CCI-NMF 的学习过程。

joint_cci_grn 的目标是:

让 GRN 的模块结构先经过有向 Joint NMF，再通过表达矩阵变成 unit-level 调控模块状态，并在同一个 Collective Joint NMF 目标中约束 CCI 的发送端与接收端 unit embedding。

因此最终的 N 不再只是 CCI 的 N，而是同时能够解释通信结构与内部调控状态的 joint-N。

4.2 符号、矩阵与维度总表

对一个时间对 $t \in \{s, t\}$ ，定义：

符号	维度	非负	生物与数学含义
$A^{(t)}$	$U_t \times U_t$	是	稀疏 CCI，有向 unit-to-unit 通信。
$W^{(t)}$	$G \times G$	是	对齐后的有向 GRN，行是 regulator gene，列是 target gene。
$X^{(t)}$	$U_t \times G$	是	unit-by-gene 表达矩阵，连接 unit 空间与 gene 空间。
$Q_{\text{reg}}^{(t)}$	$G \times r_G$	是	每个基因属于各 regulator module 的强度。
$Q_{\text{tar}}^{(t)}$	$G \times r_G$	是	每个基因属于各 target module 的强度。
S_G	$r_G \times r_G$	是	共享的 GRN 模块交互核心：regulator module a 到 target module b 的连接强度。
$M_{\text{reg}}^{(t)}$	$U_t \times r_G$	是	每个 unit 的 regulator-side 模块门控状态。
$M_{\text{tar}}^{(t)}$	$U_t \times r_G$	是	每个 unit 的 target-side 模块门控状态。
$H_{\text{send}}^{(t)}$	$U_t \times r_C$	是	最终 joint-N 的 CCI 发送端 unit 因子。
$H_{\text{recv}}^{(t)}$	$U_t \times r_C$	是	最终 joint-N 的 CCI 接收端 unit 因子。
S_C	$r_C \times r_C$	是	共享的通信模块交互核心。
B_{reg}	$r_C \times r_G$	是	把发送端 unit 因子映射到 regulator 模块状态。
B_{tar}	$r_C \times r_G$	是	把接收端 unit 因子映射到 target 模块状态。

这里 r_G 是 GRN 模块数，例如 32； r_C 是 unit latent rank，例如 32。若最终拼接发送端与接收端，希望得到 64 维特征，则设置 $r_C = 32$ 。

4.3 步骤 0：对齐 gene universe

两个时间点必须在同一 gene 顺序下构造：

$$W^{(s)}, W^{(t)} \in \mathbb{R}^{G \times G}, \quad X^{(s)} \in \mathbb{R}^{U_s \times G}, \quad X^{(t)} \in \mathbb{R}^{U_t \times G}.$$

实现规则：

1. 优先验证两个 GRN 是否已经使用同一套 2000 genes；
2. 若不完全一致，第一版使用 pairwise gene intersection，并按 gene name 重排；

3. 记录原始 gene 数、交集 gene 数、GRN 边覆盖率与 expression 覆盖率；
4. 若覆盖率低于预设阈值则报错，不静默补零后继续。

这样做是为了保证 Joint NMF 中“第 i 行/列”在两个时间点指向同一个基因，避免潜在坐标错位。

4.4 步骤 1：对 GRN 做 pairwise 有向 Joint NMF

对每个时间点的 GRN，使用：

$$W^{(t)} \approx Q_{\text{reg}}^{(t)} S_G \left(Q_{\text{tar}}^{(t)} \right)^{\top}.$$

逐项解释：

- $Q_{\text{reg}}^{(t)} \in \mathbb{R}_{\geq 0}^{G \times r_G}$ ：第 i 行表示 gene i 在各 regulator module 中的 membership；
- $Q_{\text{tar}}^{(t)} \in \mathbb{R}_{\geq 0}^{G \times r_G}$ ：第 j 行表示 gene j 在各 target module 中的 membership；
- $S_G \in \mathbb{R}_{\geq 0}^{r_G \times r_G}$ ：共享模块交互核心。 $(S_G)_{ab}$ 表示 regulator module a 指向 target module b 的总体强度；
- 之所以需要 S_G ，是因为仅用 $Q_{\text{reg}} Q_{\text{tar}}^{\top}$ 会隐含“第 a 个 regulator module 主要只连第 a 个 target module”。 S_G 允许任意 regulator module 与任意 target module 发生不同强度的有向连接；
- S_G 在时间对内共享，使两个时间点的模块交互坐标处于同一个 latent system；而 $Q_{\text{reg}}^{(t)}, Q_{\text{tar}}^{(t)}$ 保持 time-specific，允许基因在发育过程中改变模块 membership。

按元素展开：

$$W_{ij}^{(t)} \approx \sum_{a=1}^{r_G} \sum_{b=1}^{r_G} Q_{\text{reg},ia}^{(t)} (S_G)_{ab} Q_{\text{tar},jb}^{(t)}.$$

这一步把原始 $G \times G$ GRN 压缩为“基因对模块的 membership + 模块之间的有向交互”，因此保留的是模块级拓扑，而不是只保留 gene 的总入强度或总出强度。

对应的 pairwise GRN 目标为：

$$\min_{Q_{\text{reg}}^{(s)}, Q_{\text{tar}}^{(s)}, Q_{\text{reg}}^{(t)}, Q_{\text{tar}}^{(t)}, S_G \geq 0} \sum_{\tau \in \{s, t\}} \frac{\left\| W^{(\tau)} - Q_{\text{reg}}^{(\tau)} S_G (Q_{\text{tar}}^{(\tau)})^{\top} \right\|_F^2}{\left\| W^{(\tau)} \right\|_F^2 + \epsilon} + \eta_G \mathcal{R}_G.$$

分母用于消除两个时间点 GRN 总权重尺度不同造成的支配； $\mathcal{R}_G$ 可包含 L2、稀疏或列归一化正则。

4.5 步骤 2：用表达把 gene module 映射到 unit，并保留双端门控

GRN-NMF 仍在 gene 空间，而 CCI 在 unit 空间。表达矩阵 $X^{(t)}$ 是二者之间的桥梁。

对 unit u 的表达列向量 $x_u^{(t)} \in \mathbb{R}^G$ ，先计算 regulator 与 target 模块表达：

$$a_u^{(t)} = (Q_{\text{reg}}^{(t)})^{\top} x_u^{(t)} \in \mathbb{R}_{\geq 0}^{r_G},$$

$$b_u^{(t)} = (Q_{\text{tar}}^{(t)})^\top x_u^{(t)} \in \mathbb{R}_{\geq 0}^{r_G}.$$

其中 a_u 表示当前 unit 对各 regulator module 的表达支持, b_u 表示对各 target module 的表达支持。

再使用 S_G 进行模块级双端门控:

$$m_u^{\text{reg}} = a_u \odot (S_G b_u) \in \mathbb{R}_{\geq 0}^{r_G},$$

$$m_u^{\text{tar}} = b_u \odot (S_G^\top a_u) \in \mathbb{R}_{\geq 0}^{r_G}.$$

解释如下:

- $S_G b_u$ 表示每个 regulator module 所连接的 target modules 在当前 unit 中总体有多活跃;
- 再与 a_u 元素乘, 要求该 regulator module 自身也有表达支持;
- $S_G^\top a_u$ 表示每个 target module 接收到的上游 regulator-module 支持;
- 再与 b_u 元素乘, 要求该 target module 自身也有表达支持;
- 这与 gene-level $x \odot (Wx)$ 的双端门控逻辑一致, 只是把完整 gene 边压缩到可解释的 GRN module 空间。

对所有 unit 堆叠后:

$$M_{\text{reg}}^{(t)} \in \mathbb{R}_{\geq 0}^{U_t \times r_G}, \quad M_{\text{tar}}^{(t)} \in \mathbb{R}_{\geq 0}^{U_t \times r_G}.$$

至此, GRN 信息已经:

gene-gene GRN $\rightarrow$ GRN modules $\rightarrow$ unit-level gated module state.

它不需要构造任何 U 张 $G \times G$ 矩阵。

4.6 步骤 3: CCI 与 unit-level GRN state 做 Collective Joint NMF

对稀疏 CCI, 采用有向分解:

$$A^{(t)} \approx H_{\text{send}}^{(t)} S_C (H_{\text{recv}}^{(t)})^\top.$$

这里:

- $H_{\text{send}}^{(t)} \in \mathbb{R}_{\geq 0}^{U_t \times r_C}$: unit 的发送端 latent state;
- $H_{\text{recv}}^{(t)} \in \mathbb{R}_{\geq 0}^{U_t \times r_C}$: unit 的接收端 latent state;
- $S_C \in \mathbb{R}_{\geq 0}^{r_C \times r_C}$: 共享的 communication-module interaction core。

为了让同一组 unit factors 同时解释内部 GRN 状态, 增加:

$$M_{\text{reg}}^{(t)} \approx H_{\text{send}}^{(t)} B_{\text{reg}},$$

$$M_{\text{tar}}^{(t)} \approx H_{\text{recv}}^{(t)} B_{\text{tar}}.$$

其中:

$$B_{\text{reg}}, B_{\text{tar}} \in \mathbb{R}_{\geq 0}^{r_C \times r_G}.$$

B_{reg} 表示每个发送端 communication factor 与哪些 regulator modules 相关; B_{tar} 表示每个接收端 communication factor 与哪些 target modules 相关。

完整目标:

$$\begin{aligned} \min_{\Theta \geq 0} \quad & \lambda_C \sum_{\tau \in \{s, t\}} \frac{\|A^{(\tau)} - H_{\text{send}}^{(\tau)} S_C (H_{\text{recv}}^{(\tau)})^\top\|_F^2}{\|A^{(\tau)}\|_F^2 + \epsilon} \\ & + \frac{\lambda_G}{2} \sum_{\tau \in \{s, t\}} \frac{\|M_{\text{reg}}^{(\tau)} - H_{\text{send}}^{(\tau)} B_{\text{reg}}\|_F^2}{\|M_{\text{reg}}^{(\tau)}\|_F^2 + \epsilon} \\ & + \frac{\lambda_G}{2} \sum_{\tau \in \{s, t\}} \frac{\|M_{\text{tar}}^{(\tau)} - H_{\text{recv}}^{(\tau)} B_{\text{tar}}\|_F^2}{\|M_{\text{tar}}^{(\tau)}\|_F^2 + \epsilon} + \eta_C \mathcal{R}_C. \end{aligned}$$

其中 Θ 包含全部待优化非负矩阵。每个重构项除以自身输入的 Frobenius norm, 是为了避免 CCI 与 GRN 因矩阵规模、单位数或权重尺度不同而自动支配目标函数。

第一轮固定:

$$\lambda_C = \lambda_G = 1.$$

之后只有在 joint 路线有效时才做 λ_G 敏感性分析。

4.7 为什么这个目标能实现 “GRN 经过 Joint NMF”

1. 原始 GRN 先通过 $Q_{\text{reg}}, S_G, Q_{\text{tar}}$ 分解, 得到模块级有向拓扑;
2. 表达矩阵将这些 gene modules 转成每个 unit 的双端门控模块状态;
3. 同一个 H_{send} 既要重构 unit-to-unit 发送通信, 也要重构 regulator-side GRN state;
4. 同一个 H_{recv} 既要重构 unit-to-unit 接收通信, 也要重构 target-side GRN state;
5. 因此最终 unit embedding 不能只解释 CCI, 也不能只解释 GRN, 而必须找到二者共享的 latent structure。

这与先算 N、再额外拼一个 G 的路线不同: GRN 在优化阶段直接改变 N 的取值。

4.8 最终 joint-N 特征与 KL

最终每个 unit 的特征:

$$N_u^{\text{joint}} = [H_{\text{send}}(u, :), H_{\text{recv}}(u, :)] \in \mathbb{R}_{\geq 0}^{2r_C}.$$

若希望最终特征为 64 维, 设置:

$$r_C = 32, \quad 2r_C = 64.$$

之后沿用现有 feature KL:

$$p_u = \text{softmax}\left(\frac{\tilde{N}_u^{\text{joint}}}{\beta}\right), \quad C_{uv} = D_{\text{KL}}(p_u \| p_v),$$

再由 C 产生 PIJ。这里不再额外拼接一个 64 维 G ，因为 GRN 已经在 Collective Joint NMF 中进入了 N 。

4.9 一个具体形状例子

假设 spot 层:

$$U = 4000, \quad G = 2000, \quad r_G = 32, \quad r_C = 32.$$

则:

矩阵	shape	说明
A	4000×4000 sparse	稀疏 CCI。
W	2000×2000 sparse	GRN。
X	4000×2000 sparse/-dense	unit expression。
$Q_{\text{reg}}, Q_{\text{tar}}$	2000×32	gene module membership。
S_G	32×32	GRN module interaction core。
$M_{\text{reg}}, M_{\text{tar}}$	4000×32	unit GRN module states。
$H_{\text{send}}, H_{\text{recv}}$	4000×32	joint unit factors。
S_C	32×32	communication core。
$B_{\text{reg}}, B_{\text{tar}}$	32×32	communication-to-GRN mappings。
N^{joint}	4000×64	最终进入 KL 的特征。

关键点是：实现从未构造 4000 张 2000×2000 active GRN。最大的结构仍是已有的稀疏 A, W, X ，其他中间结果最多为 $U \times r$ 或 $G \times r$ 。

4.10 pairwise 语义与信息泄露控制

joint_cci_grn 只对当前 source-target 时间对执行联合分解:

$$(t_s, t_t).$$

不得把 11.5、12.5、13.5、14.5 全部放进一个全时间 joint factorization。共享的 $S_G, S_C, B_{\text{reg}}, B_{\text{tar}}$ 仅在当前时间对内共享，因此与现有 pairwise N 语义一致，不使用未来第三时间点的信息。

4.11 代码结构

新增网络 builder:

```
mignet_ce/networks/joint_cci_grn.py
```

该 builder 负责:

- 构造 sparse CCI;
- 加载并按 gene name 对齐 GRN;
- 准备 unit-by-gene expression;
- 将三个模态及其 mapping 作为 network artifacts 输出;
- 不在单时间 network builder 内强行完成跨时间 Joint NMF。

跨时间 factorization 应放在 compare feature 层, 因为它依赖一个明确的 time pair:

```
mignet_ce/pij/compare/joint_cci_grn_features.py  
mignet_ce/pij/compare/compare_joint_N_kl.py
```

推荐公共 API:

```
def fit_pairwise_grn_joint_nmf(  
    source_grn,  
    target_grn,  
    *,  
    rank_grn: int,  
    max_iter: int,  
    tol: float,  
    random_state: int,  
) -> PairwiseGrnNmfResult:  
    ...  
  
def build_unit_gated_grn_modules(  
    expression,  
    q_reg,  
    q_tar,  
    s_grn,  
) -> tuple[np.ndarray, np.ndarray]:  
    ...  
  
def fit_collective_cci_grn_nmf(  
    source_cci,  
    target_cci,  
    source_reg_modules,  
    source_tar_modules,  
    target_reg_modules,  
    target_tar_modules,  
    *,  
    rank_cci: int,  
    weight_cci: float,
```

```

weight_grn: float,
max_iter: int,
tol: float,
random_state: int,
) -> PairwiseCollectiveNmfResult:
...

```

4.12 优化与初始化要求

第一版采用交替非负最小二乘或 **multiplicative updates**，必须满足：

- 所有输入先检查非负、有限与行列对齐；
- 使用固定 random seed；
- 至少支持多次初始化，选择总 objective 最小的一次；
- 每 10 次迭代记录 CCI loss、GRN-reg loss、GRN-tar loss 与总 loss；
- 若 objective 上升超过容差，记录 warning；
- 终止条件同时考虑相对 objective 变化与 max iteration；
- 不得把 dense $U \times U$ CCI 复制多份。

5. 三个网络的实验矩阵

5.1 正式对比

网络	PIJ 方法	核心问题
light_cci	compare_N_kl	原始基线。
sparse_cci	compare_N_kl	只做 CCI 稀疏化是否改善 EI？
sparse_cci_grn	compare_N_G_block_kl	双端门控 GRN state 是否提供独立补充？
joint_cci_grn	compare_joint_N_kl	GRN 进入 Joint NMF 后是否进一步改善？

第一轮优先层级：

- spot→k150；
- spot→k40；
- k150→k40；
- gene→spot 作为控制组。

5.2 阶段化运行

1. 阶段 A：只跑 light_cci 与 sparse_cci；
2. 阶段 B：阶段 A 确认后，加入 sparse_cci_grn；
3. 阶段 C：前两步流程稳定后，再运行计算量更高的 joint_cci_grn。

每组结果都统计：

- mean EI_gain;
- median EI_gain;
- $\Pr(EI_{\text{gain}} > 0)$;
- 去除 gene 层后的同三项统计;
- 每个 layer pair 单独统计;
- PIJ 行熵与 cost 分布。

6. 配置字段建议

6.1 sparse CCI

```
cci_sparse_topk_out: int = 10
cci_sparse_topk_in: int = 10
cci_sparse_loglp: bool = True
cci_sparse_row_normalize: bool = True
cci_sparse_keep_self: bool = False
```

6.2 sparse_cci_grn

```
grn_state_topk_targets: int = 50
grn_state_dim: int = 64
grn_state_projection_seed: int = 20260713
grn_state_block_size: int = 256
compare_n_kl_weight: float = 0.5
compare_grn_kl_weight: float = 0.5
compare_n_temperature: float = <reuse current N KL beta>
compare_grn_temperature: float = <new explicit value>
```

6.3 joint_cci_grn

```
joint_grn_rank: int = 32
joint_cci_rank: int = 32
joint_weight_cci: float = 1.0
joint_weight_grn: float = 1.0
joint_grn_max_iter: int = 500
joint_collective_max_iter: int = 500
joint_tol: float = 1e-5
joint_n_init: int = 3
joint_random_state: int = 20260713
joint_min_gene_coverage: float = 0.8
```

第一轮不做大规模调参; rank 只需与当前 N 的最终特征维度保持一致。

7. 测试计划

7.1 sparse_cci

1. 人工矩阵验证 row top-k 与 col top-k 并集;
2. 验证 log1p、self-loop 和行归一化;
3. 验证空行不产生 NaN/Inf;
4. 验证 gene 层原 GRN 行为不变;
5. 验证 registry 与 CLI。

7.2 sparse_cci_grn

1. 小型 GRN 手算 $x \odot (Wx)$ 与 $x \odot (W^T x)$;
2. 验证双端门控等价于 active GRN 行和与列和;
3. 验证 top-k pruning;
4. 验证 gene-name hash projection 与列顺序无关;
5. 验证 blockwise 与一次性计算一致;
6. 验证 N/G block 分别 softmax, 最终 cost 等于加权 KL 和;
7. 验证 N 与 G 维度不同时权重仍由 λ_N, λ_G 控制。

7.3 joint_cci_grn

1. 人工小 GRN 检验 $Q_{\text{reg}} S_G Q_{\text{tar}}^T$ 的 shape 与重构;
2. 构造非对角 S_G , 验证 regulator module 可连接不同 target module;
3. 验证 gene-name 对齐与低覆盖率报错;
4. 手算模块表达 a, b 与模块级双端门控 $a \odot (S_G b)$ 、 $b \odot (S_G^T a)$;
5. 验证 M_{reg} 只约束 H_{send} , M_{tar} 只约束 H_{recv} ;
6. 验证各 loss 项归一化后再按权重求和;
7. 验证固定 seed 与多初始化选择;
8. 验证最终 N^{joint} shape 为 $U \times 2r_C$;
9. 验证只使用当前 time pair, 不读取第三时间点;
10. 验证 PIJ 非负、行随机、无 NaN/Inf。

7.4 集成 smoke test

先只跑:

```
organ=heart
time_points=11.5,12.5
layer_pair=seurat_k40:seurat_k150
```

依次覆盖四组:

- light_cci + compare_N_kl;

- `sparse_cci + compare_N_kl`;
- `sparse_cci_grn + compare_N_G_block_kl`;
- `joint_cci_grn + compare_joint_N_kl`。

8. Codex 不得做的事情

1. 不得删除或覆盖原始 `light_cci`;
2. 不得把三个新增网络合并为同一个网络名称下的开关;
3. 不得把 gene-gene GRN 边压成 `unit self-loop` 后与 CCI 相加;
4. 不得把 h5ad 原始 `regulon-TF` 列冒充为当前 GRN 特征;
5. 不得把双端表达门控替换为纯 Wx 或 $W^T x$ 传播;
6. 不得在 `sparse_cci_grn` 中对 N/G 拼接后做一次统一 `softmax`;
7. 不得在 `joint_cci_grn` 中省略 S_G 或把它默认成单位阵;
8. 不得在不同时间点不按 `gene name` 对齐矩阵;
9. 不得进行全时间 `joint factorization`;
10. 不得默认构造或保存每个 `unit` 的完整 $G \times G$ active GRN;
11. 不得在未记录各项 `loss` 与 `convergence` 的情况下输出 `joint-N`;
12. 不得第一轮边改结构边大规模调权重。

9. 完成标准

- 三个新增网络均在 `registry` 中有独立名称: `sparse_cci`、`sparse_cci_grn`、`joint_cci_grn`;
- `sparse_cci` 输出完整稀疏化诊断;
- `sparse_cci_grn` 明确保留 `gene-level` 双端表达门控;
- N/G 采用分块 KL, 维度不同不会自动改变总权重;
- `joint_cci_grn` 中 S_G 、所有 `factor` 与映射矩阵的维度均可追溯;
- GRN 确实先经过 `pairwise directed Joint NMF`;
- GRN module 通过 `expression bridge` 转成 `unit-level gated state`;
- CCI 与 GRN state 共同约束 `Collective Joint NMF`;
- 最终 `joint-N` 可直接进入现有 KL 流程;
- 全部 PIJ 非负、行随机、无 NaN/Inf;
- 每组均输出 `mean/median EI_gain` 与正增益比例;
- 单元测试与 `smoke test` 全部通过。

10. 最终算法一句话

先以 `sparse_cci` 单独验证 CCI 过密是否造成通信特征同质化；再以 `sparse_cci_grn` 保留 `gene-level` 双端表达门控，将 GRN 变成 `unit-level 64` 维状态并在分块 KL 层与 CCI-NMF 融合；最后以 `joint_cci_grn` 将 GRN 压成有向调控模块，通过表达构造 `unit-level gated module state`，并让这些状态与稀疏 CCI 共同学习 `joint-N`，随后沿用 KL 生成 `PIJ` 与计算 `EI_gain`。